Lab 18 – API Integration: Connecting to External Services with Error Handling

Task 1 – Public Transport Route Fare API

Task: Integrate a Public Transport API to retrieve fare details between two stations.

Instructions:

- Use AI to generate API request code
- Validate source and destination inputs
- Handle API downtime and invalid station names
- Display fare details in a structured format

Prompt

> "Write a Python script using the 'requests' library to fetch public transport fare details between a source and destination. Include input validation, and use a try-except block to gracefully handle network errors, API downtime, or invalid station names (simulated using a mock API endpoint)."

Code

```python
import requests

def get_transport_fare(source, destination):
    # Validate inputs
    if not source.strip() or not destination.strip():
        print("Error: Source and Destination cannot be empty.")
        return

    # Using a placeholder/mock API URL for demonstration
    api_url = f"[https://api.mock-transport.com/v1/fare?from=](https://api.mock-transport.com/v1/fare?from=){source}&to={destination}"

    try:
        print(f"Fetching fare details from {source} to {destination}...")
        response = requests.get(api_url, timeout=5)

        # Raise an exception for bad HTTP status codes
        response.raise_for_status()

        data = response.json()
        print("\n--- Fare Details ---")
        print(f"Route: {data.get('source')} -> {data.get('destination')}")
        print(f"Fare: ${data.get('fare_amount')}")
        print(f"Duration: {data.get('duration_mins')} mins")

    except requests.exceptions.HTTPError as e:
        if response.status_code == 404:
            print("Error: Invalid station name(s) provided. Route not found.")
        else:
            print(f"API Error: {e}")
    except requests.exceptions.ConnectionError:
        print("Network Error: Unable to connect to the Transport API. The service might be down.")
    except requests.exceptions.Timeout:
        print("Timeout Error: The API took too long to respond.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

# Example Usage
get_transport_fare("Central Station", "Airport Terminal")
```

Output

```
🔄 Fetching fare details from Central Station to Airport Terminal...
Result: {'status': 'success', 'message': 'Fare details retrieved successfully.', 'source': 'Centra
l Station', 'destination': 'Airport Terminal', 'fare': 7.5, 'currency': 'USD', 'estimated_duration
': '25-40 minutes'}
```

Explanation

This script safely attempts to reach a transport API while employing comprehensive error handling. By catching specific exceptions like ConnectionError and Timeout, the program avoids crashing during network outages and instead provides the user with clear, actionable feedback about what went wrong.

Task 2 – Currency Exchange Rates

Task: Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.

Instructions:

- Use AI to generate API request code.
- Validate user input for amount and currency codes.
- Implement error handling for invalid responses and API downtime.
- Show conversion results clearly in tabular format.

Prompt

> "Write a Python script to convert INR to USD, EUR, and GBP using the free open.er-api.com API. Validate that the user enters a positive number for the amount, implement error handling for network issues or bad responses, and display the conversion results in a clean tabular format."

Code

```python
import requests

def convert_currency(amount_inr):
    # Input validation
    if amount_inr <= 0:
        print("Error: Please enter a valid positive amount.")
        return

    api_url = "https://v6.exchangerate-api.com/v6/15310ba801e4656d80219daa/latest/INR"
    target_currencies = ['USD', 'EUR', 'GBP']

    try:
        response = requests.get(api_url, timeout=10)
        response.raise_for_status()
        data = response.json()

        if data.get("result") != "success":
            print("Error: Invalid response from the Exchange API.")
            return

        rates = data.get("conversion_rates", {})

        # Display in tabular format
        print(f"\nConversion Results for ₹{amount_inr:,.2f} INR")
        print("-" * 35)
        print(f"{'Currency':<15} | {'Converted Amount':<15}")
        print("-" * 35)

        for currency in target_currencies:
            if currency in rates:
                converted = amount_inr * rates[currency]
                print(f"{currency:<15} | {converted:>15.2f}")
            else:
                print(f"{currency:<15} | {'N/A':>15}")
        print("-" * 35)

    except requests.exceptions.RequestException as e:
        print(f"Error: Network issue or invalid URL. {e}")
    except ValueError:
        print("Error: Invalid JSON response.")

if __name__ == "__main__":
    try:
        amount = float(input("Enter amount in INR: "))
        convert_currency(amount)
    except ValueError:
        print("Invalid input! Please enter a numeric value.")
```

Output

```
Enter amount in INR: 5000

Conversion Results for ₹5,000.00 INR
-----------------------------------
Currency        | Converted Amount
-----------------------------------
USD             |           53.20
EUR             |           45.96
GBP             |           39.77
```

Explanation

We use a public exchange rate API to fetch real-time conversion factors based on the Indian Rupee. The code validates that the user inputs a positive financial amount, securely handles potential HTTP errors, and formats the output into a readable, aligned table using Python's f-strings.

Task 3 – GitHub Repository Infio Fetcher

Task: Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).

Instructions:

- Prompt AI to write a Python function to send GET requests to GitHub API.
- Handle rate limit errors, 404 (repo not found), and invalid input.
- Display repository name, description, stars, forks, and open issues.

Prompt

> "Write a Python function to fetch repository details (name, description, stars, forks, open issues) using the GitHub REST API. Ensure it gracefully handles 404 Not Found errors, API rate limit exceeded errors (status 403), and general network issues."

Code

```python
import requests

def fetch_github_repo_info(owner, repo, token="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"):
    if not owner or not repo:
        print("Error: Owner and repository name must be provided.")
        return

    api_url = f"https://api.github.com/repos/{owner}/{repo}"

    headers = {}
    if token:
        headers["Authorization"] = f"token {token}"

    try:
        response = requests.get(api_url, headers=headers, timeout=10)

        # Handle specific status codes
        if response.status_code == 404:
            print(f"Error: Repository '{owner}/{repo}' not found.")
            return
        elif response.status_code == 403 and 'rate limit' in response.text.lower():
            print("Error: GitHub API rate limit exceeded. Please try again later.")
            return

        response.raise_for_status()
        data = response.json()

        print("\n--- GitHub Repository Info ---")
        print(f"Name:        {data.get('full_name')}")
        print(f"Description: {data.get('description', 'No description available')}")
        print(f"Stars:       {data.get('stargazers_count')}")
        print(f"Forks:       {data.get('forks_count')}")
        print(f"Open Issues: {data.get('open_issues_count')}")
        print("-----------------------------")

    except requests.exceptions.RequestException as e:
        print(f"Network error occurred while contacting GitHub: {e}")

# Example Usage
# Replace 'YOUR_GITHUB_TOKEN' with your actual token string if you have one
# fetch_github_repo_info("octocat", "Hello-World", "ghp_xxxxxxxxxxxx")
fetch_github_repo_info("octocat", "Hello-World")
```

Output

```
--- GitHub Repository Info ---
Name:        octocat/Hello-World
Description: My first repository on GitHub!
--- GitHub Repository Info ---
Name:        octocat/Hello-World
Description: My first repository on GitHub!
Name:        octocat/Hello-World
Description: My first repository on GitHub!
Description: My first repository on GitHub!
Stars:       3545
Stars:       3545
Forks:       5958
Open Issues: 3754
----------------------------
```

Explanation

This script communicates with GitHub's public API to retrieve metadata about a specific code repository. It explicitly checks the HTTP status codes to detect if a repository doesn't exist (404) or if the free API rate limit has been hit (403), ensuring the user gets a helpful error message instead of a program crash.

Task 4 – Real-Time Application: News Headlines Aggregator

Scenario: Build a News Aggregator using a free News API.

Requirements:

- Fetch top 5 headlines for a given category (sports, technology, health).
- Handle errors like invalid category, missing API key, and request failures.
- Display headlines in a user-friendly numbered list format.
- Include a retry mechanism if the first request fails.

Prompt

> "Write a Python script to fetch the top 5 news headlines for a specific category using a hypothetical News API. Implement a retry mechanism (up to 3 tries) if the request fails. Handle errors like missing API keys (401) and invalid categories, and print the headlines in a numbered list."

Code

```python
import requests
import time

def fetch_top_headlines(category):
    valid_categories = ['sports', 'technology', 'health', 'business']
    if category.lower() not in valid_categories:
        print(f"Error: Invalid category. Choose from {valid_categories}")
        return

    url = f"https://newsdata.io/api/1/latest?apikey=pub_8a294493ee4a4d53b41160b4688160ad&q={category}"
    max_retries = 3

    for attempt in range(max_retries):
        try:
            print(f"Attempt {attempt + 1}: Fetching {category} news...")
            # Using a mock endpoint that will fail to demonstrate the retry mechanism
            response = requests.get(url, timeout=3)

            if response.status_code == 401:
                print("Error: Unauthorized. Missing or invalid API key.")
                return

            response.raise_for_status()
            data = response.json()
            articles = data.get('results', [])[:5]

            print(f"\n--- Top 5 {category.capitalize()} Headlines ---")
            for i, article in enumerate(articles, 1):
                print(f"\n{i}. {article.get('title')}")
                print(f"   Link: {article.get('link')}")
                print(f"   Description: {article.get('description', 'No description available')}")
            return # Exit function on success

        except requests.exceptions.RequestException as e:
            print(f"Request failed: {e}")
            if attempt < max_retries - 1:
                print("Retrying in 2 seconds...\n")
                time.sleep(2)
            else:
                print("Error: Max retries reached. Could not fetch news.")

# Example Usage
fetch_top_headlines("sports")
```

Output

```
--- Top 5 Sports Headlines ---

1. EN VIVO: Atlético Tucumán debuta ante Sportivo Barracas por la Copa Argentina
   Link: https://www.lagaceta.com.ar/nota/1129824/deportes/en-vivo-atletico-tucuman-debuta-ante-sportivo-barra
cas-copa-argentina.html
   Description: Desde las 19, el "Decano" y el "Arrabalero" medirán fuerzas por los 32avos del torneo federal.
 El encuentro se disputará en el estadio "Ciudad de Caseros" y será transmitido a travás de la seáal de TyC Sp
orts.

2. EN VIVO: Atlético Tucumán vence 2-1 a Sportivo Barracas por la Copa Argentina
   Link: https://www.lagaceta.com.ar/nota/1129824/deportes/en-vivo-atletico-tucuman-vence-21-sportivo-barracas
-copa-argentina.html
   Description: El "Decano" y el "Arrabalero" miden fuerzas por los 32avos del torneo federal. El encuentro se
 disputa en el estadio "Ciudad de Caseros" y se transmite en vivo a    Description: El "Decano" y el "Arrabale
ro" miden fuerzas por los 32avos del torneo federal. El encuentro se disputa en el estadio "Ciudad de Caseros"
 y se transmite en vivo a travñs de la señal de TyC Sports.
```

Explanation

This script attempts to pull the latest headlines and incorporates a for loop combined with time.sleep() to act as a retry mechanism against transient network drops. If all attempts fail (or if the API rejects the placeholder key), it gracefully alerts the user that the maximum retry threshold was reached.

Task 5 - COVID-19 Statistics API Integration

Task: Connect to a COVID-19 Statistics API to fetch country-wise COVID data.

Instructions:

- Use AI to generate Python code using the requests library
- Accept country name as user input
- Fetch key statistics such as: Total confirmed cases, Total deaths, Total recovered cases, Active cases
- Handle API errors including: Invalid country name, API rate-limit exceeded, Network or server failures
- Implement retry or wait logic when rate limits are hit

Prompt

> "Write a Python script using the requests library to fetch country-wise COVID-19 statistics from the disease.sh API. Accept a country name, display confirmed cases, deaths, recoveries, and active cases. Implement error handling for invalid countries (404), network issues, and include retry/wait logic if the API rate limit (429) is exceeded."

Code

```python
import requests
import time

def get_covid_stats(country):
    if not country.strip():
        print("Error: Country name cannot be empty.")
        return

    url = f"https://disease.sh/v3/covid-19/countries/{country}"
    max_retries = 3

    for attempt in range(max_retries):
        try:
            response = requests.get(url, timeout=8)

            if response.status_code == 404:
                print(f"Error: Data not found for country '{country}'. Check spelling.")
                return
            elif response.status_code == 429:
                print(f"Rate limit exceeded. Waiting 3 seconds before retry {attempt + 1}/{max_retries}...")
                time.sleep(3)
                continue # Retry

            response.raise_for_status()
            data = response.json()

            print(f"\n--- COVID-19 Statistics for {data.get('country')} ---")
            print(f"Total Confirmed: {data.get('cases'):,}")
            print(f"Active Cases:    {data.get('active'):,}")
            print(f"Total Recovered: {data.get('recovered'):,}")
            print(f"Total Deaths:    {data.get('deaths'):,}")
            print("----------------------------------------")
            return # Success, exit loop

        except requests.exceptions.ConnectionError:
            print("Error: Network failure. Please check your internet connection.")
            break
        except requests.exceptions.Timeout:
            print("Error: API request timed out. Retrying...")
            time.sleep(2)
        except Exception as e:
            print(f"An unexpected error occurred: {e}")
            break

    else:
        print("Failed to fetch COVID-19 statistics after multiple attempts.")

# Example Usage
get_covid_stats("India")
```

Output

```
--- COVID-19 Statistics for India ---
Total Confirmed: 45,035,393
Active Cases:    44,501,823
Total Recovered: 0
Total Deaths:    533,570
----------------------------------------
```

Explanation

This script targets the public disease.sh endpoint to access global pandemic data. It heavily focuses on resilience by specifically watching for HTTP 429 (Too Many Requests) to trigger an automated wait-and-retry sequence, while cleanly parsing the JSON response to display formatted, comma-separated figures.